

Git_Guide

Somya Verma

The guide

Create a new RStudio Project:

A project was created with the name Assignment2_CRP and a QMD file was made named **exmaple.qmd** and was rendered to the HTML format. As seen in [Figure 1](#)

Initialise this folder as a git repository: Commnad Line

The following steps were taken in order to push the local repository to the remote and initialising Git through the command line. The commit can be seen in [Figure 2](#)

```
cd .Assignment2_CRP
```

Changed into the project directory.

```
git init
```

Made a Git repository through local on GitHub repository. It creates a hidden `.git` directory which will track the files. Below is the first initial commit.

```
git status
```

Checked the current repository status and tracked files.

```
git add .
```

Staged all project files for commit.

```
git commit -m "Adding QMD and R Project files"
```

Created a commit containing the QMD and R project files.

```
git push
```

Uploaded local commits to the GitHub repository.

Git_Guide

AUTHOR
Somya Verma

The guide

Create a new RStudio Project:

A project was created with the name Assignment2_CRP and a QMD file was made named **exmaple.qmd**

Figure 1: Rendered__file

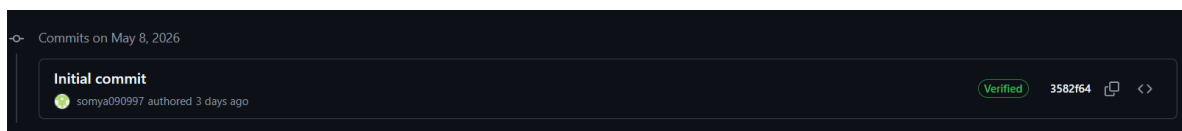


Figure 2: Initialising Git from Command Line

Local files Pushed to Remote Git Repository

Below in figure Figure 3 the files pushed from the local folder is reflected with the `git commit` message pushed through the command line.

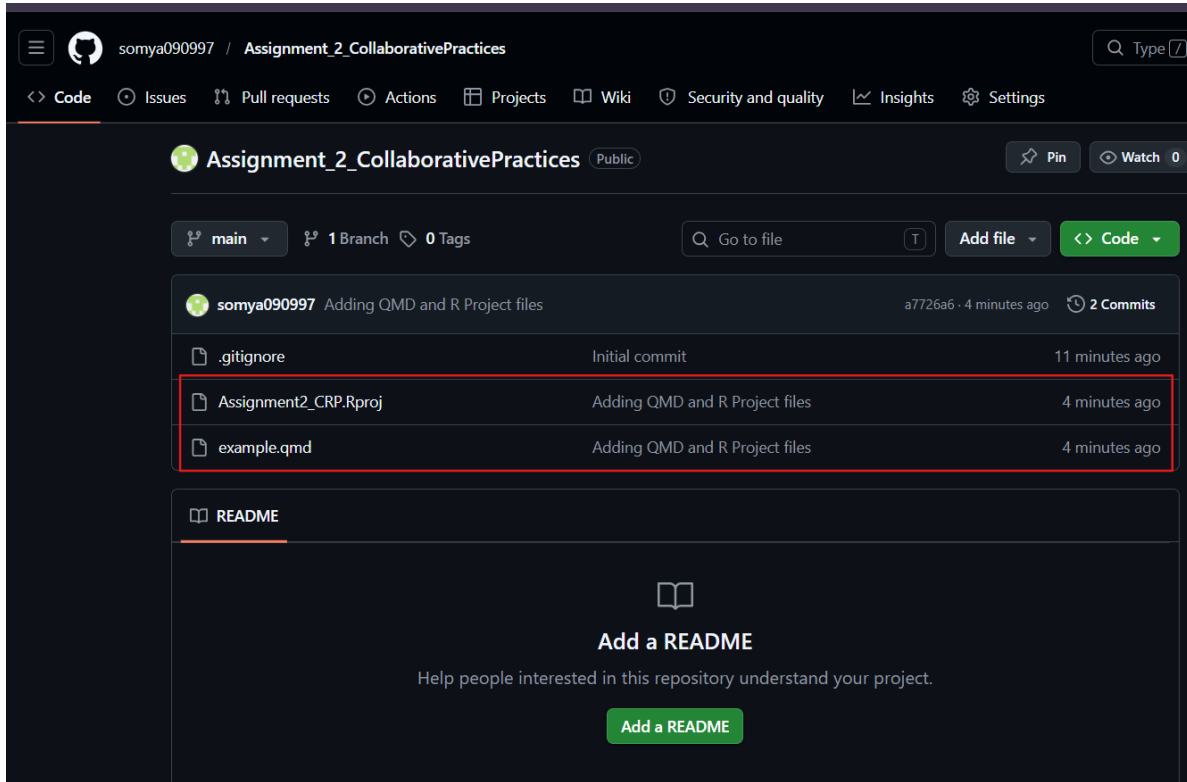


Figure 3: Pushing Local To Remote

Adding a New Branch: testbranch

With help of the `git checkout -b testbranch` we not only create a new branch but also switch. The alternative to this can be `git switch -c testbranch`. As shown in the Figure 4

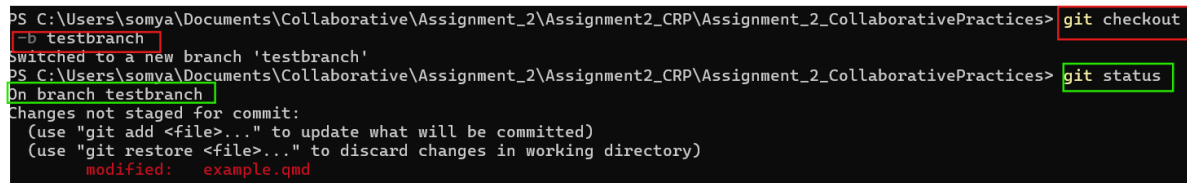


Figure 4: Creating New Branch: Testbranch

What issues can come while creating a branch? You can refer to the below picture Figure 5 If you used the general `git push`, it will not work and will not show your commit. As for the first time, the branch as to be established with an upstream. Hence we will have to use `git push -u origin testbranch` or we can also try `git push --set-upstream origin testbranch`.

Post this step, you can do the general `git push` command to push changes in the testbranch.

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push -u origin testbranch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 445 bytes | 445.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'testbranch' on GitHub by visiting:
remote:   https://github.com/somya090997/Assignment_2_CollaborativePractices/pull/new/testbranch
remote:
To github.com:somya090997/Assignment_2_CollaborativePractices.git
 * [new branch]      testbranch -> testbranch
branch 'testbranch' set up to track 'origin/testbranch'.
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push
Everything up-to-date
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git status
On branch testbranch
Your branch is up to date with 'origin/testbranch'.
```

Figure 5: Adding New Branch: Testbranch

Adding a folder named “data”

Using the command `mkdir` a folder named `data` was added that can be seen in the below Figure 6 can be seen. This folder had the csv file from Assignment1.

The regular `git add`, `git commit` and `git push` commands were used to make changes reflect in remote repository as well. It can be seen in Figure 7

Note: Always check which branch you are working on or making the folder through `git branch`

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git branch
* main
  testbranch
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> mkdir data

Directory: C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices

Mode                LastWriteTime         Length Name
----                -
d-----          8/05/2026   2:19 PM             data

PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git add data
```

Figure 6: Creating a folder: Data

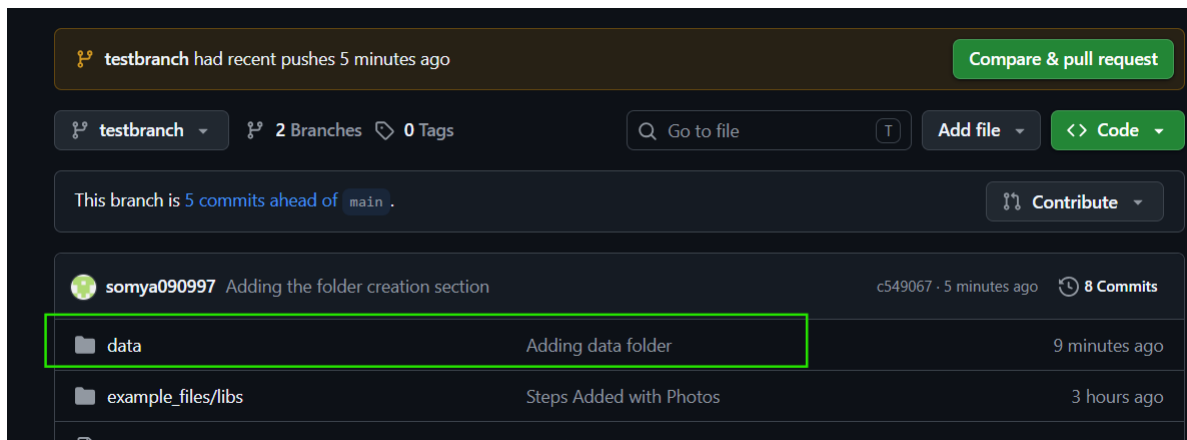


Figure 7: Adding a folder: Data

Amending the commit

We will check our git log of commits with help of `git log --online`. In the below Figure 8 you will see `git rebase -i HEAD~3` has been used to locate the commit to amend the commit message from “Adding Folder” (seen in Figure 7) to “*Adding data folder with csv data*”.

Then `git commit --amend` is used to change the commit message within VIM. Once the message is edited, save it by typing `:wq` that saves the changes and gets back to the command line.

Why do we use `git push --force`? After using `git commit --amend`, the commit hash changed because Git created a new version of the commit. A normal push was rejected since the remote repository still contained the old commit history. Therefore, `git push --force` was required to overwrite the remote history with the amended commit.

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git rebase -i HEAD~3
Stopped at 10fdc7f... # Including folder amending section
You can amend the commit now, with

    git commit --amend

Once you are satisfied with your changes, run

    git rebase --continue
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git commit --amend
[detached HEAD ade7881] Section of adding folder and amending commit
Date: Fri May 8 14:30:50 2026 +1000
1 file changed, 2 insertions(+)
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push --force
```

Figure 8: Amending Commit

Check the Figure 9 to see the change in comment.

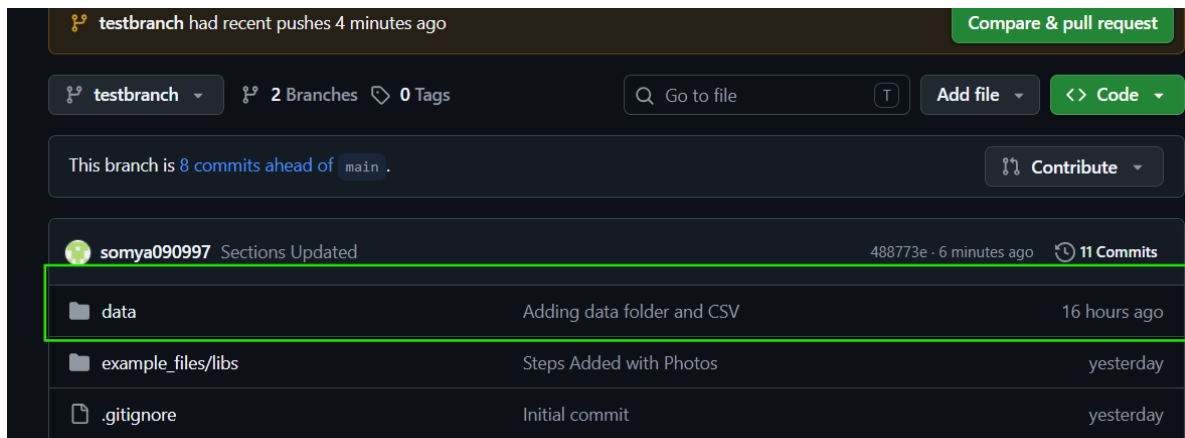


Figure 9: Amended Commit

Switching To Main To Create Conflict

Below in Figure 10 we can see that we are now working. As we make changes in the `main` branch it will cause conflict with the other `testbranch`

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git checkout
main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git branch
* main
testbranch
```

Figure 10: Switching to main

Creating and Solving a merge conflict

In the below Figure 11 you can see that the conflict has been created in the `example.qmd` file. The Figure 12 shows the same message in the command line.

In the below Figure 13 you can see that the conflict have been resolved and the branch has been merged as well.

You can also see the merging through the git Graph below.

Tagging v1.0 on main

Once the branch is merged on Main, the tag is added to that commit with help of the command `git tag -a v1.0 -m` in the command line as in Figure 15. You can see in Figure 16 that it has been added, in @fig you can see it is reflected in the github repository as well.

```
commit message pushed through the command line.
51
52 ![Pushing Local To Remote](PushingtoGit.png){#fig-PushingtoGit}
53
54 ## Adding a New Branch: testbranch
55
56 k<<<<<< HEAD
57 ## Switching To Main To Create Conflict
58
59 Below in @fig-Switchingtomain we can see that we are now working. As we make changes in the 'main
60 branch' it will cause conflict with the other **testbranch**
61
62 ![Switching to main](Switching from to main.png){#fig-Switchingtomain}
63
64 =====
65 With help of the 'git checkout -b testbranch' we not only create a new branch but also switch. The
66 alternative to this can be 'git switch -c testbranch'. As shown in the @fig-CreatingBranch
67
68 ![Creating New Branch: Testbranch](Creating_Branch.png){#fig-CreatingBranch}
69
70 What issues can come while creating a branch? You can refer to the below picture @fig-Addabbranch If
71 you used the general 'git push', it will not work and will not show your commit. As for the first
72 time, the branch as to be established with an upstream. Hence we will have to use 'git push -u
73 origin testbranch' or we can also try 'git push --set-upstream origin testbranch'.
```

Figure 11: Conflict in QMD

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push
Everything up-to-date
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git merge testbranch
Auto-merging example.html
CONFLICT (content): Merge conflict in example.html
Auto-merging example.qmd
CONFLICT (content): Merge conflict in example.qmd
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices>
```

Figure 12: Conflict message in CL

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git add .
warning: in the working copy of 'example.html', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git commit -m "testbranch merged"
[main b98390e] testbranch merged
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 115.28 KiB | 498.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:somya090997/Assignment_2_CollaborativePractices.git
 16a7c29..b98390e main -> main
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git status
On branch main
Your branch is up to date with 'origin/main'.
```

Figure 13: Resolving and Merging

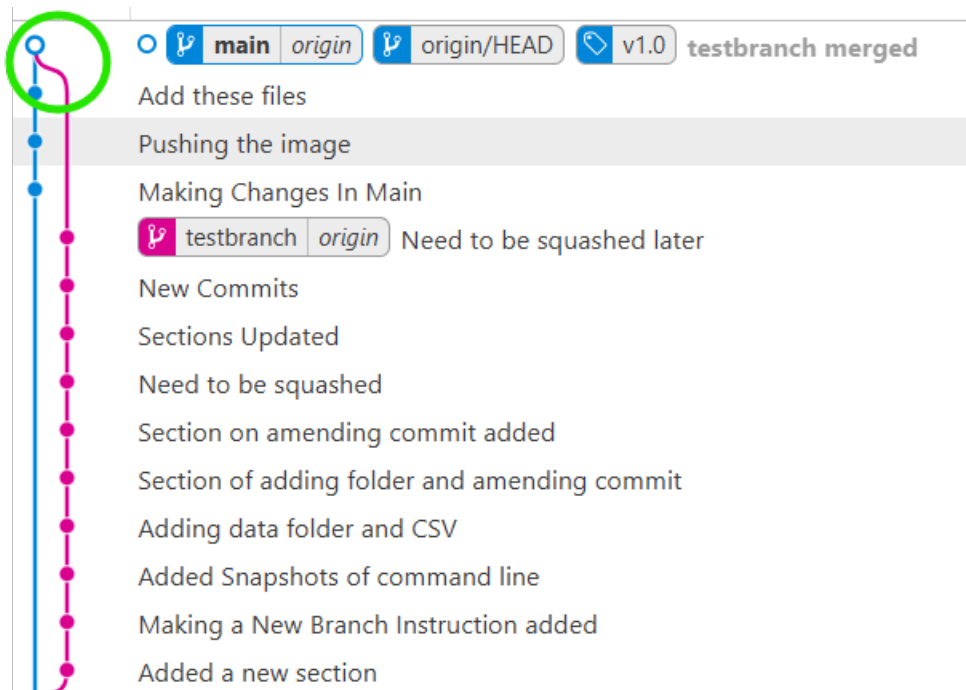


Figure 14: Git Graph

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git tag -a v
1.0 -m "Version 1.0 Tag"
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices>
```

Figure 15: Terminal Tag

Graph	Description	Date	Author	Commit
○	Uncommitted Changes (4)	10 May 2026 1...	*	*
○	main <i>origin</i> origin/HEAD adding new images	10 May 2026 1...	somya verma	5fa7bee9
	Updating OMD content	9 May 2026 10:...	somya verma	0eddeb05
●	v1.0 testbranch merged	9 May 2026 09:...	somya verma	b98390eb
	Add these files	9 May 2026 09:...	somya verma	16a7c29a
	Pushing the image	9 May 2026 09:...	somya verma	8870bad6
	Making Changes In Main	9 May 2026 09:...	somya verma	dade3522
●	testbranch <i>origin</i> Need to be squashed later	9 May 2026 09:...	somya verma	c8a32db1
	New Commits	9 May 2026 09:...	somya verma	96f46e14
	Sections Updated	9 May 2026 09:...	somya verma	488773e4
	Need to be squashed	8 May 2026 16:...	somya verma	69e79655
	Section on amending commit added	8 May 2026 15:...	somya verma	c6304441
	Section of adding folder and amending commit	8 May 2026 14:...	somya verma	ade78815
	Adding data folder and CSV	8 May 2026 14:...	somya verma	f4e25f2b

Figure 16: Git Graph Tag

The tags can be seen in the github repository. As seen in the Figure 17

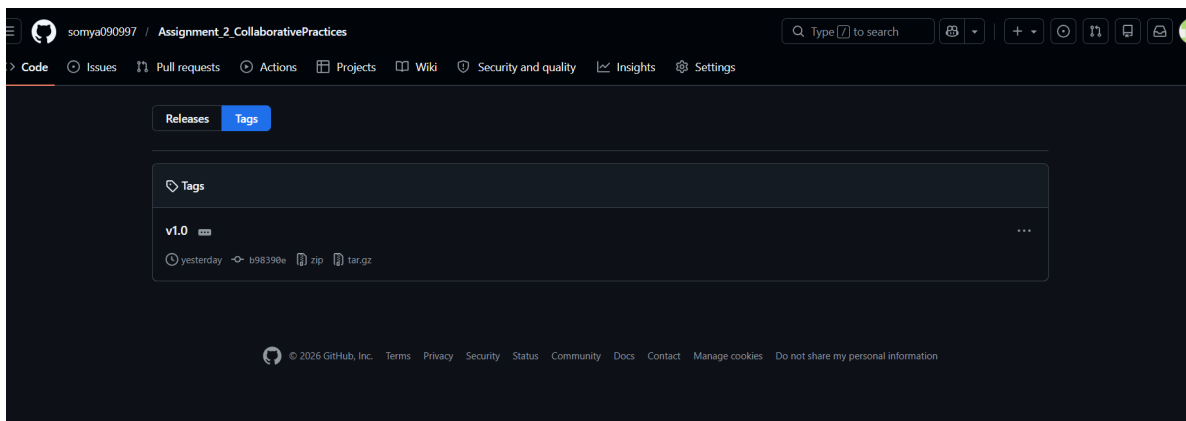


Figure 17: Github Tag

Deleting the test branch

To delete the test branch, make sure you are **NOT** still working on it. Switch from the testbranch to main. Use the command `git branch -d testbranch` to get it deleted from the local. Now, inorder to get away with the branch on the remote, push the changes with `git push origin --delete testbranch`. You can see the same in Figure 18 and Figure 19

```

PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git commit -m "Deleting Test Branch"
[main 06e8002] Deleting Test Branch
5 files changed, 78 insertions(+)
create mode 100644 Resolving_and_merging.png
create mode 100644 Tag_Github.png
create mode 100644 tag_commit.png
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git push origin --delete testbranch
To github.com:somya090997/Assignment_2_CollaborativePractices.git
- [deleted]    testbranch
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices>

```

Figure 18: Github Tag

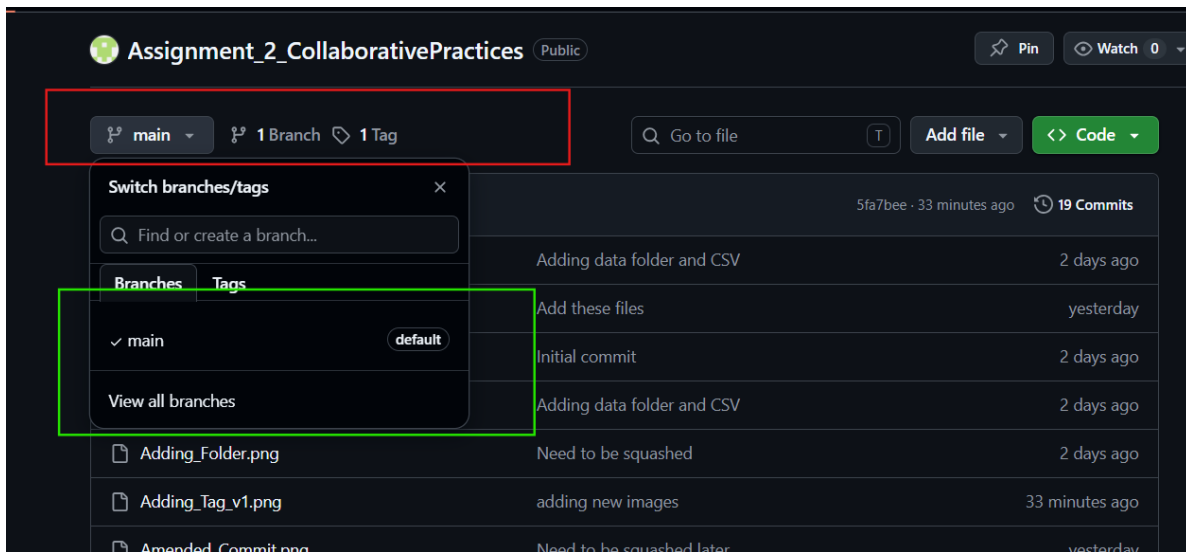


Figure 19: Github Branch deleted

Commit log in condensed form

Below is the condensed form of the commits done using the command `git log --oneline --graph --decorate --all`. It shows not only the history of commits but also a map of merge history and the branch structure.

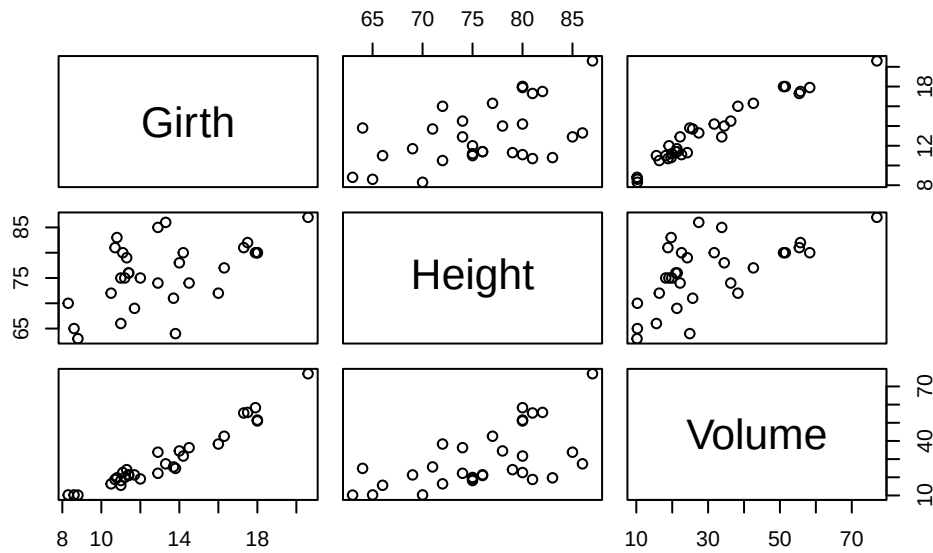
```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git log --oneline --graph --decorate --all
* 06e8002 (HEAD -> main) Deleting Test Branch
* 5fa7bee (origin/main, origin/HEAD) adding new images
* 0eddeb0 Updating QMD content
* b98390e (tag: v1.0) testbranch merged
| \
| * c8a32db Need to be squashed later
| * 96f46e1 New Commits
| * 488773e Sections Updated
| * 69e7965 Need to be squashed
| * c630444 Section on amending commit added
| * ade7881 Section of adding folder and amending commit
| * f4e25f2 Adding data folder and CSV
| * 42e448c Added Snapshots of command line
| * a02810d Making a New Branch Instruction added
| * 2598ad2 Added a new section
| * | 16a7c29 Add these files
| * | 8870bad Pushing the image
| * | dade352 Making Changes In Main
| /
* 5975ff9 Steps Added with Photos
* a7726a6 Adding QMD and R Project files
* 3582f64 Initial commit
```

Figure 20: Condensed Commit history

Simple Plot and Maintaining Local Changes

The following plot uses the built-in `cars` dataset.

```
plot(trees)
```



A new section containing a simple plot was added to `example.qmd`. The changes were staged and committed using `git commit` and `git add` and then pushed. It was then checked with help of `git log` as seen in Figure 21

```
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CHP\Assignment_2_CollaborativePractices> git log --oneline
7c26d8e (HEAD -> main, origin/main, origin/HEAD) Add simple plot section
96e8002 Deleting Test Branch
5fa7bee adding new images
0eddeb0 Updating QMD content
b98390e (tag: v1.0) testbranch merged
16a7c29 Add these files
8870bad Pushing the image
dade352 Making Changes In Main
c8a32db Need to be squashed later
96f46e1 New Commits
488773e Sections Updated
69e7965 Need to be squashed
c630444 Section on amending commit added
ade7881 Section of adding folder and amending commit
f4e25f2 Adding data folder and CSV
42e448c Added Snapshots of command line
a02810d Making a New Branch Instruction added
2598ad2 Added a new section
5975ff9 Steps Added with Photos
a7726a6 Adding QMD and R Project files
3582f64 Initial commit
```

Figure 21: Adding Commit for Plot

The above commit was undone with help of `git reset`. The `--soft` option removed the commit from the Git history while preserving all local changes and keeping them staged.

After running the reset command, the repository status was checked. The output showed that `example.qmd` was still modified and staged for commit, confirming that the commit had been

```

PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git reset --soft HEAD~1
PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Condensed_log.png
        new file:   command_line_delete.png
        new file:   deletion_of_branch.png
        modified:   example.html
        modified:   example.qmd
        new file:   example_files/figure-html/unnamed-chunk-1-1.png

```

Figure 22: Git Reset Used

removed without losing any local work. Additionally, `git log --oneline` no longer displayed the *Add simple plot section* commit.

```

PS C:\Users\somya\Documents\Collaborative\Assignment_2\Assignment2_CRP\Assignment_2_CollaborativePractices> git log --oneline
06e8002 (HEAD -> main) Deleting Test Branch
5fa7bee adding new images
0eddeb0 Updating QMD content
b98390e (tag: v1.0) testbranch merged
16a7c29 Add these files
8870bad Pushing the image
dade352 Making Changes In Main
c8a32db Need to be squashed later
96f46e1 New Commits
488773e Sections Updated
69e7965 Need to be squashed
c630444 Section on amending commit added
ade7881 Section of adding folder and amending commit
f4e25f2 Adding data folder and CSV
42e448c Added Snapshots of command line
a02810d Making a New Branch Instruction added
2598ad2 Added a new section
5975ff9 Steps Added with Photos
a7726a6 Adding QMD and R Project files
3582f64 Initial commit

```

Figure 23: Local Changes Maintained